IE4-DSL PREPERATION SHEET

Chapter 1

Synchronous Serial Communication with an Audio Codec

Stisys Group (please ☒)	Team members
01 ☐ 02 ☐ 03 ☐	Mehria Rahmani
Team (please ☒)	Mateus Hretzko Gritzenco
A ☐ B ☐ C ☐ D ☐ E ☐ F ☐	Laksha Mugon

Prep Task 1

If you use the system clock from the oscillator of the Modsys 2.0 board (f_{SYSCLK} = 100 MHz), you obviously cannot exactly set a sampling frequency 48 kHz, because the ratio of the two frequencies is not exactly a power of two.

- What sampling frequency $\hat{f}_{\mathrm{LRC}}$ can you set that is closest to f_{LRC}=48 kHz?
- Calculate the relative error $\epsilon_{\mathrm{rel}} = (f_{\mathrm{LRC}} - \hat{f}_{\mathrm{LRC}})/f_{\mathrm{LRC}}$.
- What ratio $f_{\mathrm{CLK}}/f_{\mathrm{SYSCLK}}$ did you assume?

Set sampling frequency: 48,828125 KHz

Relative error:-1,73%

Ratio f_CLK / f_SYSCLK = 8

Prep Task 2

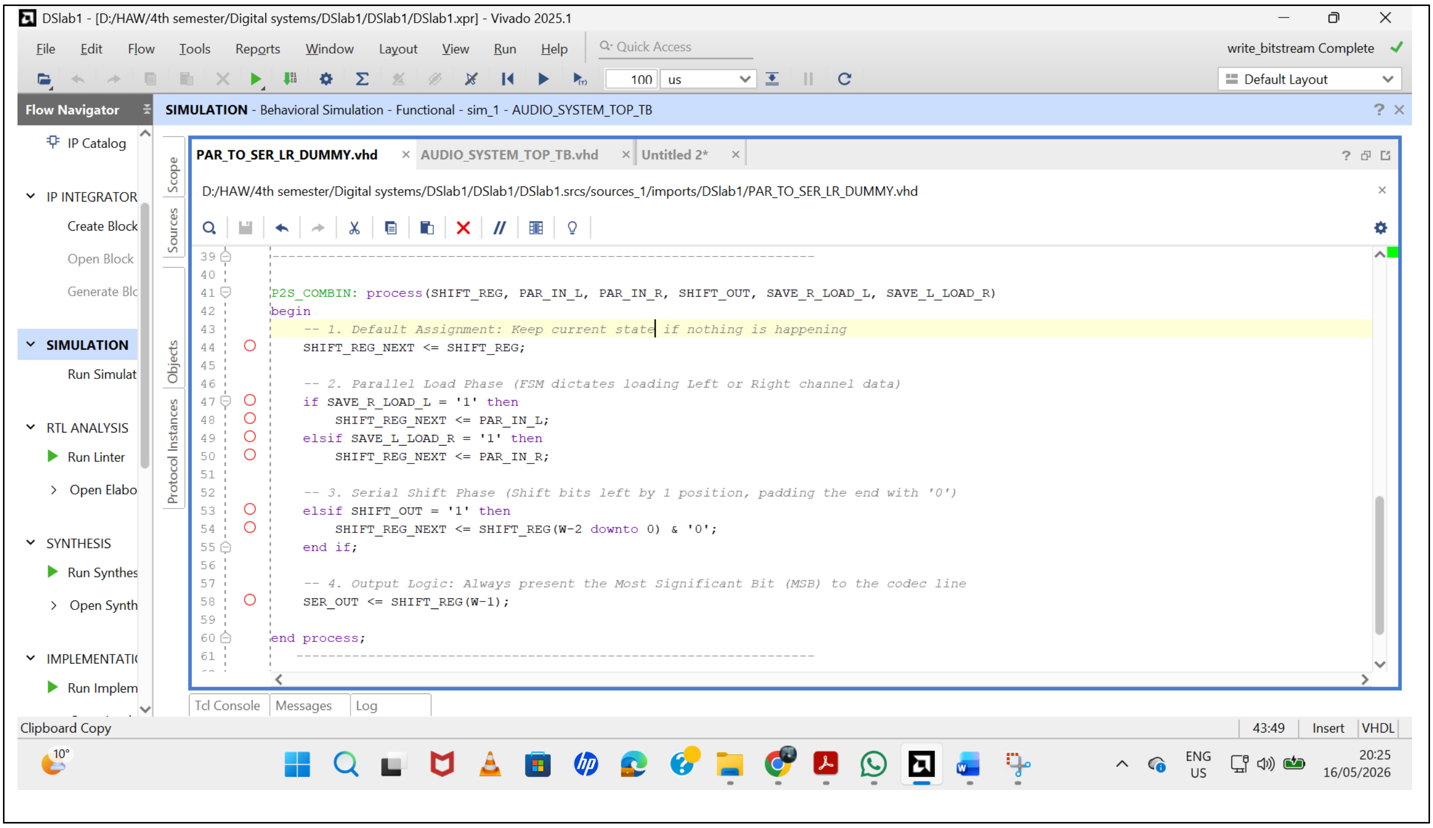
Design a behavioral model in VHDL of the PAR_TO_SER_LR parallel-to-serial converter and verify it.

- First, simulate the overall system as it was provided to you – incomplete yet, but ready for simulation. If you succeed in the simulation without errors, start modeling the parallel-to-series converter.
- Use the file PAR_TO_SER_LR_DUMMY.vdh as starting point and use the the block diagram from the lecture as a guideline for implementation.
- Please mind, the module should have only one shift register for both channels and no parallel registers.
- Write 3 (non-trivial) test cases that will give you confidence that your parallel-to-serial converter will work. One test case
  - has a meaningful title (for your manager who is responsible for the project) and
  - a detailed description of the scenario (for your intern who is writing the testbench).
  - This includes describing what you need to observe in the waveform viewer to know that the test case has passed.
- Simulate at least these 3 test cases and copy per test case a relevant part of your ModelSim simulation (waveforms) into the preparation sheet, based on which you explain why the test passed (if necessary why you think not).
  Please note: Always simulate your module in the overall system. It is duplicate work to write a testbench only for the parallel-to-serial converter, because a good testbench already exists. Better adapt this one appropriately.

Congratulations if you got this far in your preparation. That was quite a bit of work and you had to familiarize yourself with VHDL modeling and tools. Now you can look calmly at the lab session.

VHDL CODE

Enter the VHDL code of your parallel-to-serial converter here.



TEST CASES

A test case consists of a short, concise title (for the project manager) and a detailed description of how to implement this case (for the HAW intern).

<table>
<tr><td rowspan="2">Test case 1</td><td>Concise and precise title (max. 10 words):

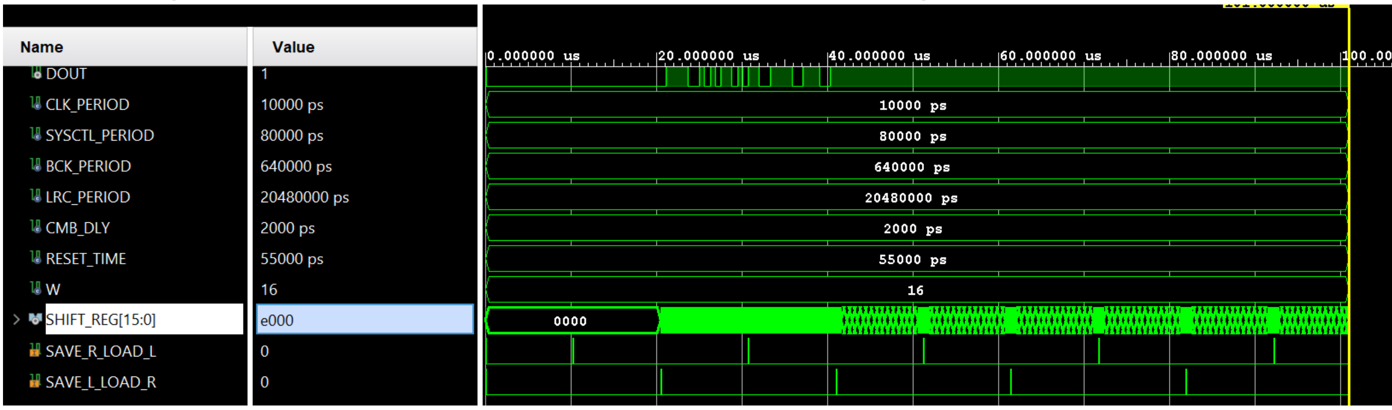
Power-On Reset and Initial Register State Verification</td></tr>
<tr><td>Description (detailed enough to let someone else implement the test case in the test bench):
Assert the active-low reset signal (SRESETN) to '0' at time 0 ns and maintain it low for at least 5.5 micro sec while running the master system clock (CLK) to check the circuit's boot-up behavior. Following this period, drive SRESETN back high ('1') to release the system from reset</td></tr>
<tr><td rowspan="2">Test case 2</td><td>Title: Priority Parallel Loading of Dual-Channel Audio Samples</td></tr>
<tr><td>Description: Static 16-bit test values, such as 0xFE00, are applied to the left and right parallel audio input buses (PAR_IN_L and PAR_IN_R). The top-level finite state machine (FSM) then generates single-cycle control pulses using the signals SAVE_R_LOAD_L and SAVE_L_LOAD_R at the beginning of each left and right audio frame. These pulses command the shift register to load the corresponding 16-bit audio sample for serial transmission.</td></tr>
</table>

Test case 3	**Title:** Sequential MSB-First Shifting and Output Serialization
	Description: The simulation is executed for at least 100 µs to allow a complete audio transmission sequence to occur. During this period, the bit clock (BCK) continues toggling normally, while the SHIFT_OUT control pulse is activated at the required bit intervals. This causes the stored parallel audio data inside the shift register to shift outward serially through the output line bit-by-bit.

SIMULATION RESULTS

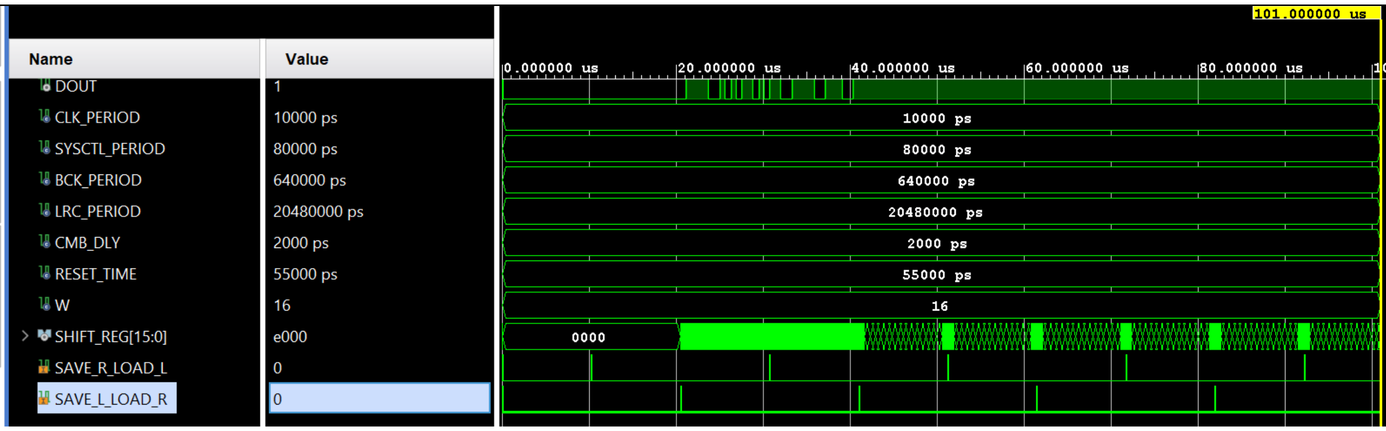
Please mind again to **only simulate the overall system,** i.e. include the parallel-to-serial converter in your overall system and use the provided test bench for simulation. Adapt it to your needs, i.e. to your 3 test cases. Paste in the table below the relevant waveform with which you can prove that your test cases passed successfully.

Test case 1	Waveform (Relevant screenshot of waveforms from ModelSim)



	Explanation, why successful From 0 µs to approximately 5.5 µs, the internal signal SHIFT_REG remains fixed at the hexadecimal value 0000. This confirms that the synchronous reset logic inside the P2S_REGS process operates correctly. During this period, the active-low reset signal (SRESETN) is asserted low, causing the shift register to initialize to a known zero state before normal operation begins.
Test	Waveform (Relevant screenshot of waveforms from ModelSim)

case 2

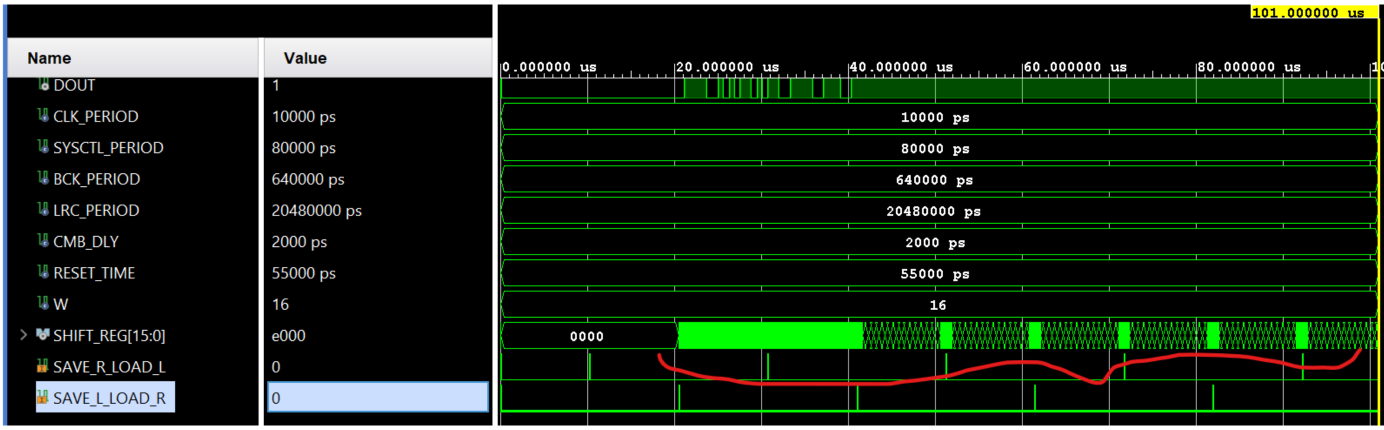


Explanation, why successful

At approximately 5.5 µs, the SAVE_R_LOAD_L control signal becomes high ('1') for one clock cycle. On the next rising edge of the system clock, the internal signal SHIFT_REG changes from 0000 to the hexadecimal test value FE00. This confirms that the parallel loading logic correctly loads the complete 16-bit audio sample into the shift register before the serial shift-ing process begins.

Test case 3

Waveform (Relevant screenshot of waveforms from ModelSim)



Explanation, why successful

After the parallel load operation, from approximately 25 µs to 100 µs, the signal SHIFT_REG shows continuous activity, indicating that its internal values are changing on every bit-clock cycle. At the same time, the DOUT signal switches between high and low states as bits are shifted out serially from the register. This confirms that the shift logic correctly moves the data through the register and transmits the Most Significant Bit (MSB) to the serial output line, creating a continuous serial data stream.